MinIO Metadata & Access Analytics Service 架構設計（ClickHouse-only，5B Objects）

Oct 17, 2026 · @Jason Lin

1. 摘要與目標

本系統以 Kafka + ClickHouse 為唯一的狀態元件，收集所有 MinIO cluster 的 metadata 變更與所有經過 gateway 的存取事件。這些資料同時用於兩件事：object 層級的 access 分析，用來驅動 hot ↔ warm ↔ cold 的 tiering 決策（核心目標）；以及 S3 listing 加速（單頁 p99 < 500 ms，冷大 prefix ≥ 50x）。資料 I/O 完全不經過本系統，MinIO 仍是唯一的 source of truth。

現況問題

- **Listing 慢**：MinIO 沒有集中式 metadata DB，`ListObjects` 必須對 erasure set 內所有 drive 做 directory walk 再 merge sort，成本約與「prefix 下 object 數 × drive 數」成正比。5B objects、LOSF 的情況下，冷 prefix listing 常達秒級以上。
- **沒有 access 記錄**：沒有任何地方記錄 object 最後一次被讀的時間，tiering 只能依年齡或人工判斷，結果是仍在使用的資料被搬到 cold，而長期無人讀的資料佔著 hot。
- **沒有全域視圖**：多個 MinIO cluster 之間無法回答「hot tier 上有多少 bytes 已 90 天未讀」這類問題。

目標

目標	衡量方式	目標值
Object 層級 access profile	每個 object 的 last access、7 / 30 / 90 天讀取次數與 bytes	涵蓋 100% object，分析延遲 < 15 min
Tiering 決策自動化	每日產出 demote / promote 候選，由 SAL Static Tiering 執行	Mis-tier 率 < 2%、ping-pong 率 < 0.5%
冷大 prefix listing	ListObjectsV2 單頁 latency，對比 MinIO baseline	≥ 50x，p99 < 500 ms
全量 enumeration	1 億 keys prefix 匯出時間	≥ 50x，分鐘級
容量報表	任意 bucket / prefix 的 object 數、bytes、tier 分佈	秒級

設計原則

1. **Out-of-band**：本系統只觀察與決策，不進入 PUT / GET 的 critical path；任何故障的最壞結果是 listing 退回 MinIO 原生速度、tiering 暫停。

2. **單一事件入口**：所有狀態變更都以事件進入 Kafka，再由 ClickHouse 消費；bootstrap、校正、tiering 回流都走同一套規則，可重放、可重建。
3. **分析優先**：資料模型以 `(cluster_id, bucket, object_key)` 為排序主軸，讓 5B 筆 metadata 與數十億筆 access 事件能以主鍵順序 join。
4. **決策與執行分離**：本系統只產出 tiering 決策，搬移由 SAL Static Tiering 以 immutable rule 執行，兩邊各自可審計、可回滾。

非目標：取代 MinIO data path、strong consistency listing、高 QPS（> 1K page/s）線上 listing、POSIX namespace。

2. 需求與假設

下表的讀寫量為假設值，Phase 0 需以 MinIO audit log 與 OpenResty access log 校正；其中 GET / HEAD 的 peak 速率直接決定 ClickHouse 儲存規模，是最需要先確認的數字。

Functional requirements

1. **Metadata 同步（F2）**：所有 MinIO cluster 的建立、覆寫、刪除、storage class 變更，以 eventual consistency 同步到 ClickHouse。
2. **Access 收集（F3）**：所有經過 gateway 的 GET / HEAD / PUT / DELETE / LIST，含 bucket、key、bytes、status、principal。
3. **Access profile（F6）**：每個 object 的 first / last access、7 / 30 / 90 天讀取次數與 bytes、目前 tier 與停留天數。
4. **Tiering 決策（F6）**：依 policy 與成本模型每日產出 demote / promote 候選，支援 dry-run、審核、限流、回滾。
5. **執行閉環（F6）**：候選交由 SAL Static Tiering 執行，執行結果以事件回流，用於衡量 mis-tier 率。
6. **S3-compatible listing（F4）**：`ListObjectsV2`（prefix、`delimiter=/`、start-after、continuation-token、max-keys），既有 client 零改動。
7. **擴充查詢（F4）**：prefix 統計、tier 分佈、條件查詢、access profile 查詢、大 prefix 匯出。
8. **初始化與校正（F5）**：新 bucket bootstrap、持續 reconcile、空目錄清理。

Non-functional requirements

項目	假設 / 目標
Object 總數	5B，年成長 30–50%，設計上限 20B
平均 key 長度	約 120 bytes
Mutation（PUT + DELETE）	平均 20K ops/s，peak 100K ops/s

項目	假設 / 目標
Read（GET + HEAD）	平均 50K ops/s，peak 200K ops/s（待確認）
Listing QPS	平均 50、peak 500 page/s
Listing latency	單頁 p50 < 150 ms、p99 < 500 ms
Metadata freshness	p99 < 60 s；24 h 內最終收斂
Access analytics freshness	p99 < 15 min
Tiering 決策週期	每日一次，可手動觸發
保留期	Raw access event 30 天；每日聚合 400 天
可用性	Listing 99.9%（可 fallback）；分析 99.5%
Durability	Metadata 可由 MinIO 重建；access 歷史不可重建，需 2 replica + 異地備份

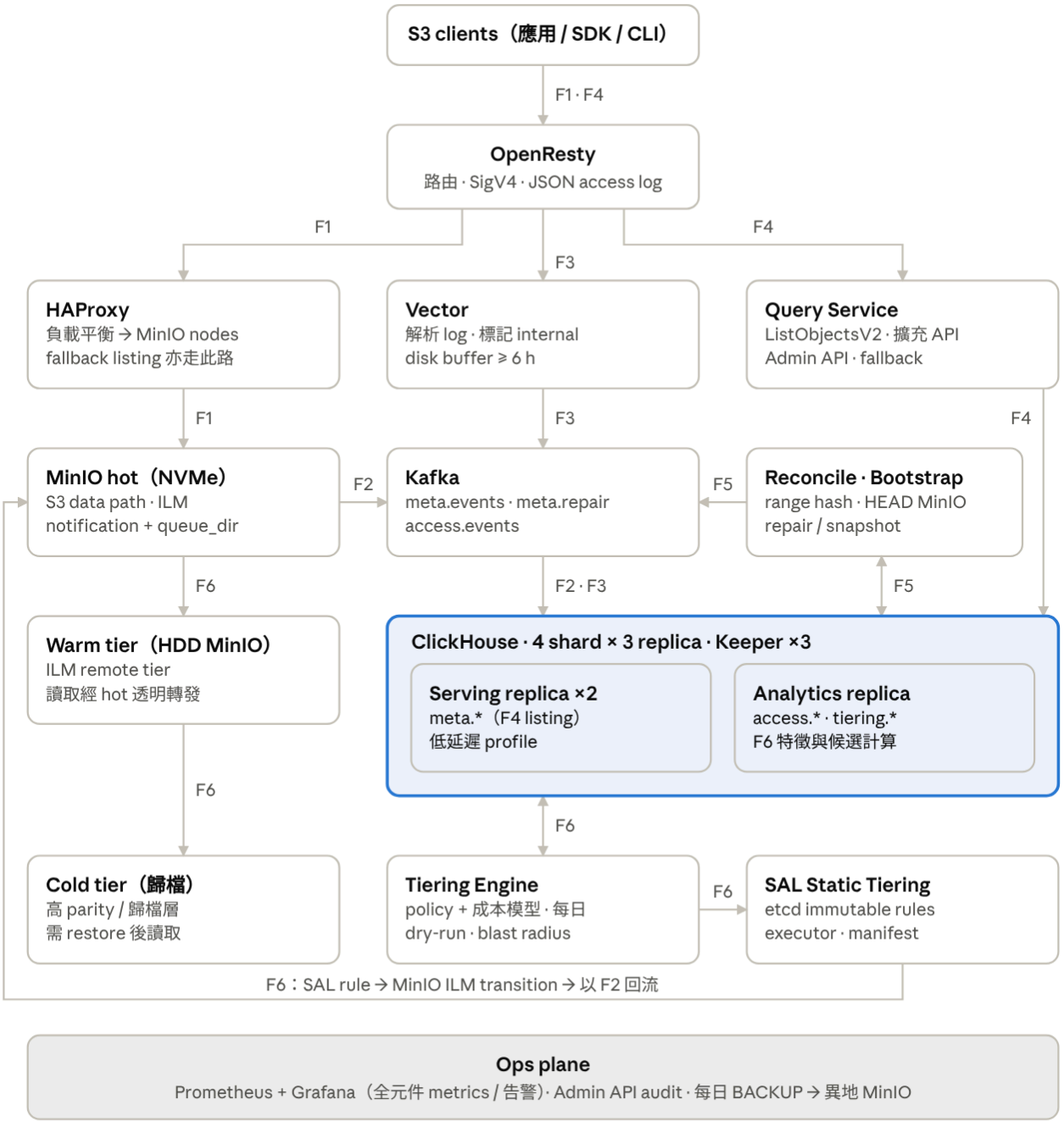
限制與既有技術棧

- 多個 MinIO cluster；前方已有 OpenResty → HAProxy，access log 與 listing 路由都在這層處理。
- Warm / cold tier 以 MinIO ILM remote tier 實作，client 永遠透過 hot cluster 存取，被搬走的 object 由 MinIO 透明轉發或需 restore。
- Tiering 執行沿用 SAL Static Tiering（etcd config store、append-only immutable rules）。
- MinIO CE 已 archive，設計只依賴 bucket notification、ILM 與 S3 API，可換 fork 或其他 S3 後端。
- ClickHouse 為自建 on-prem 叢集（ClickHouse Keeper），不依賴 ClickHouse Cloud 功能；鎖定 LTS 版本。

3. 完整架構與資料流

系統分成四層：入口層（OpenResty、HAProxy、Vector）、儲存層（MinIO hot / warm / cold）、事件與分析層（Kafka、ClickHouse）、決策與執行層（Query Service、Reconciler、Tiering Engine、SAL）。六條資料流以 F1–F6 標示在箭頭旁，細節見第 4 節。

所有事件匯入 ClickHouse，同時驅動 listing 與 tiering 閉環



F1 資料 I/O · F2 metadata 變更 · F3 access 事件 · F4 listing · F5 bootstrap / 校正 · F6 tiering 閉環

完整架構 · 15 個元件、F1–F6 六條資料流

讀圖方式：左欄是資料本身的位置（F1 進 hot，F6 驅動 hot → warm → cold）；中欄是事件匯流（F2、F3 經 Kafka 進 ClickHouse）；右欄是使用 ClickHouse 的服務（F4 listing、F5 校正）；底部的回流線是 F6 閉環：SAL 下發的 rule 在 MinIO 觸發 transition，再以 F2 事件更新 metadata。

元件	層	職責	參與的資料流	狀態
OpenResty	入口	路由 listing 與 data I/O、輸出 JSON access log	F1、F3、F4	無
HAProxy	入口	負載平衡到 MinIO 節點；listing fallback	F1、F4	無
Vector	入口	解析 access log、標記 internal principal、送 Kafka	F3	Disk buffer
MinIO hot	儲存	Source of truth、S3 data path、ILM、bucket notification	F1、F2、F5、F6	資料
Warm / Cold tier	儲存	ILM remote tier，經 hot 透明存取或 restore	F1、F6	資料
Kafka	事件	`meta.events`、`meta.repair`、`access.events` 的緩衝與重放	F2、F3、F5	保留 3 天
ClickHouse serving replica	分析	`meta.*` 的低延遲查詢	F2、F4	主要狀態
ClickHouse analytics replica	分析	`access.*`、`tiering.*`、特徵與候選計算	F3、F5、F6	主要狀態
ClickHouse Keeper	分析	Replicated 表協調	F2、F3	Quorum
Query Service	服務	S3 listing shim、擴充 API、Admin API	F4	無
Reconcile / Bootstrap	服務	初始全量、range 比對、repair、空目錄清理	F5	無
Tiering Engine	決策	套用 policy 與安全限制，輸出 rule 與 manifest	F6	結果存 ClickHouse
SAL Static Tiering	執行	以 immutable rule 觸發 MinIO transition	F6	etcd
Prometheus + Grafana	Ops	所有元件的 metrics 與告警	全部	TSDB
異地備份 MinIO	Ops	Access 與 tiering database 的每日備份	—	備份檔

4. 資料流詳解

六條資料流中，只有 F1 是線上資料路徑且完全不經過本系統；F2、F3 是兩條事件匯入，F4、F6 是兩條使用 ClickHouse 的出口，F5 負責初始化與最終正確性。所有寫入 metadata 的路徑都以同一套 `ver` 規則（4.2）合併，因此可以重疊、重放而不衝突。

資料流	路徑	量（peak）	延遲目標	交付語意
F1 資料 I/O	Client → OpenResty → HAProxy → MinIO hot（→ warm / cold）	依業務	不變	MinIO 原生
F2 Metadata 變更	MinIO notification → Kafka `meta.events` → ClickHouse `meta.*`	100K events/s	p99 < 60 s	At-least-once，`ver` 去重
F3 Access 事件	OpenResty log → Vector → Kafka `access.events` → ClickHouse `access.*`	200K events/s	p99 < 15 min	At-least-once，可能略多計
F4 Listing	Client → OpenResty → Query Service → ClickHouse serving replica	500 page/s	p99 < 500 ms	Bounded staleness
F5 Bootstrap / 校正	MinIO listing → Parquet / repair → ClickHouse	背景、限流	24 h 內收斂	以 HEAD 確認
F6 Tiering 閉環	ClickHouse → Tiering Engine → SAL → MinIO ILM → F2 回流	每日 batch	1 個決策週期	決策可審計、可回滾

4.1 F1 — 資料 I/O（不經過本系統）

1. Client 以 SigV4 送出 PUT / GET / DELETE 到 OpenResty。
2. OpenResty 依路由規則轉給 HAProxy，HAProxy 負載平衡到 MinIO hot cluster 節點。
3. 已被 transition 到 warm / cold 的 object，GET 時由 MinIO 透明轉發到 remote tier，或回傳需 restore 的錯誤（cold）。
4. OpenResty 在回應後寫一筆 JSON access log，這是 F3 的來源；寫 log 不影響 request latency。

4.2 F2 — Metadata 變更

1. 每個 MinIO cluster 設定 Kafka notification target，訂閱 `s3:ObjectCreated:*`、`s3:ObjectRemoved:*`、`s3:ObjectTransition:*`，並啟用 `queue_dir`，Kafka 不可用時事件暫存在 MinIO 節點本機。Transition 事件在使用中的 MinIO 版本上是否完整發出，Phase 0 需實測。

2. Topic `meta.events.<cluster>`，partition key = hash(bucket, object_key)。
3. ClickHouse 以 Kafka table engine `meta.kafka_events` 消費，正規化 MV 寫入 `meta.object_meta`；`object_meta` 上再掛兩個 MV 寫入 `object_by_parent` 與 `dir_children`。所有路徑（stream、bootstrap、repair）都只寫 `object_meta` 這一個入口。
4. 事件對應與 `ver` 規則：

來源	寫入內容	`ver`
`ObjectCreated:*`（Put、Copy、CompleteMultipartUpload）	完整 metadata，`is_deleted = 0`	事件時間 `eventTime`（ns）
`ObjectTransition:Complete`	更新 `storage_class`	事件時間
`ObjectRemoved:Delete`、`DeleteMarkerCreated`	`is_deleted = 1`	事件時間
Bootstrap snapshot（F5）	完整 metadata	object 的 `mtime`
Reconciler repair（F5，經 HEAD 確認）	完整 metadata 或 tombstone	HEAD 觀測時間

用事件時間而非到達時間當 `ver`，亂序到達的事件仍能正確排序。Bootstrap 刻意用較小的 `mtime`，listing 期間若物件剛被刪除，delete 的 `ver` 會勝出，不會讓物件復活；repair 用 HEAD 時間，因為 HEAD 是強一致讀。

5. Kafka engine 設定：`kafka_max_block_size = 1048576`、`stream_flush_interval_ms = 7500`（大批次、少 part）、`kafka_handle_error_mode = 'stream'`（壞訊息寫入 `meta.kafka_errors`，不阻塞 consumer）。
6. **Late event**：正規化 MV 將事件時間早於 now() − 24 h 的事件改寫入 `meta.late_events`，由 F5 以 HEAD 確認後再修正。這個門檻與 tombstone 至少保留 7 天（8.3）配合，確保任何被接受的舊事件都還有 tombstone 可比較。

4.3 F3 — Access 事件

1. OpenResty 寫 JSON access log（bucket、key、op、status、bytes、range、principal、request_id），由同機的 Vector agent 讀取。不在 `log_by_lua` 直接送 Kafka，以免 Kafka 抖動影響 request path。
2. Vector 解析 path-style / virtual-host URL 取出 bucket 與 key，標記 internal principal（Reconciler、SAL executor、replication、備份），以 disk buffer（≥ 6 h）送入 Kafka `access.events.<site>`。
3. ClickHouse 以 Kafka engine 消費，MV 寫入 `events_raw`、`object_daily`、`object_profile`、`prefix_daily`；internal 事件只進 `events_raw`，不進聚合表，否則 Reconciler 的定期 HEAD 會讓所有 object 看起來都是熱的。

4. At-least-once 造成的少量重複只會讓讀取次數略為偏高，偏差方向是「較晚 demote」，屬於安全的一側。
5. Access 歷史無法回補：MinIO 不記錄 access time，`object_profile` 只能從 F3 上線那天開始累積，因此 F3 必須最先上線（第 12 節）。

4.4 F4 — Listing

1. OpenResty 判斷 `GET /<bucket>?list-type=2`（及 v1），bucket 狀態為 `SERVING` 且 delimiter 為空或 `/` 時轉給 Query Service，否則走 F1 到 MinIO。
2. Query Service 驗證 SigV4 與 `s3:ListBucket` 權限，依 sharding key 找到 bucket 所屬 shard，對 serving replica 執行固定模板 SQL（第 6 節）。
3. 回傳標準 `ListBucketResult` XML，附 `x-mds-watermark` 與 `x-mds-source`。
4. ClickHouse 異常、lag 超標或 client 要求 strong consistency 時，Query Service 經 HAProxy 改向 MinIO 原生 listing。

4.5 F5 — Bootstrap 與校正

1. **Bootstrap**：先啟動 F2 消費並記下 T0（capture first）；依 top-level prefix 切片平行 listing（每 cluster 限流），結果寫成 Parquet 到 MinIO staging bucket；以 `INSERT INTO meta.object_meta SELECT … FROM s3('…/staging/<bucket>/*.parquet')` 匯入，衍生表由 MV 自動產生；完成後 bucket 轉為 `SERVING`。
2. **Range 比對**：所有 (bucket, prefix range) 輪替，每個 range 每 24 h 至少比對一次。ClickHouse 端以 `groupBitXor(cityHash64(object_key, etag, size))` 算 range hash，MinIO 端以 listing 算同樣的 hash，不一致才逐筆 diff。5B objects / 24 h 約需 58K keys/s 的 listing 吞吐，排在各 cluster 離峰時段。
3. **修正**：差異 key 先 HEAD 確認，再以 repair 事件送入 Kafka `meta.repair`，與 F2 走同一條 ingest 路徑。
4. **優先順序**：late event、`kafka_errors`、近期 drift 率高的 bucket 優先。
5. **空目錄清理**：每日從 `object_meta FINAL` 算出仍有存活 object 的目錄，對 `dir_children` 中已無存活 object 者寫入 tombstone，`ver` 設為掃描開始時間減 1 h，避免誤刪掃描期間剛建立的目錄。

4.6 F6 — Tiering 閉環

1. 每日在 analytics replica 上重建 `tiering.object_features`，套用 policy 與成本模型產生 `tiering.candidates`（第 7 節）。
2. Tiering Engine 將候選合併為 prefix rule 與 object manifest，套用 blast radius 限制後，寫入 SAL Static Tiering 的 etcd（immutable rule）或交給 SAL executor。
3. SAL 在 MinIO hot cluster 上觸發 ILM transition，object 移到 warm / cold（或 promote 回 hot）。
4. MinIO 發出 `ObjectTransition` 事件經 F2 回流，更新 `object_meta.storage_class`；SAL 執行紀錄寫入 `tiering.actions`，用於計算 mis-tier 率與 rollback。

5. ClickHouse 資料模型

資料分成三個 database：`meta` 存「現在有哪些 object」（ReplacingMergeTree 處理 LWW），`access` 存「誰在何時讀了什麼」（append-only MergeTree 加聚合表），`tiering` 存特徵、候選與執行紀錄。所有表以 `(cluster_id, bucket, object_key)` 為排序主軸並以同一個 sharding key 分片，tiering join 全部在 shard 內完成。

表	Engine	ORDER BY	寫入來源	用途	保留
`meta.object_meta`	ReplacingMergeTree(ver, is_deleted)	cluster_id, bucket, object_key	F2、F5	主表；無 delimiter listing；tiering join	永久（tombstone 7 天後清除）
`meta.object_by_parent`	ReplacingMergeTree(ver, is_deleted)	cluster_id, bucket, parent, object_key	MV（object_meta）	`delimiter=/` 的 Contents	同上
`meta.dir_children`	ReplacingMergeTree(ver, is_deleted)	cluster_id, bucket, parent, child_prefix	MV + 每日清理	`delimiter=/` 的 CommonPrefixes	同上
`meta.late_events` / `meta.kafka_errors`	MergeTree	cluster_id, bucket, ts	F2	交給 F5 處理	7 天
`access.events_raw`	MergeTree	cluster_id, bucket, ts	F3	稽核、重算	30 天
`access.object_daily`	AggregatingMergeTree	cluster_id, bucket, object_key, day	MV	Object 每日讀取	400 天
`access.object_profile`	AggregatingMergeTree	cluster_id, bucket, object_key	MV	Object 終身 first / last access	隨 object 存活
`access.prefix_daily`	AggregatingMergeTree	cluster_id, bucket, prefix, day	MV	Prefix 熱度報表	400 天
`tiering.object_features`	MergeTree	cluster_id, bucket, object_key	F6 每日	決策特徵	最近 2 次 run
`tiering.candidates` / `tiering.actions`	MergeTree	run_id, cluster_id, bucket, object_key	F6	決策輸出、執行紀錄	400 天

5.1 `meta.object_meta`

```
CREATE TABLE meta.object_meta ON CLUSTER mds
(
    cluster_id     LowCardinality(String),
    bucket         LowCardinality(String),
    object_key     String,
```

```
    parent         String,                  -- 直屬目錄，'a/b/c.txt' → 'a/b/'
    size           UInt64,
    etag           FixedString(16),         -- binary MD5；multipart 另存字串欄
位
    mtime          DateTime64(3),
    version_id     String,
    storage_class  LowCardinality(String),  -- hot / warm / cold
    content_type   LowCardinality(String),
    user_tags      Map(String, String),
    ver            UInt64,                  -- 規則見 4.2
    is_deleted     UInt8,
    ingest_ts      DateTime DEFAULT now()
)
ENGINE = ReplicatedReplacingMergeTree(
    '/clickhouse/tables/{shard}/meta/object_meta', '{replica}', ver,
is_deleted)
PARTITION BY (cluster_id, cityHash64(bucket) % 16)
ORDER BY (cluster_id, bucket, object_key)
SETTINGS index_granularity = 1024;
```

- 同一 key 只保留最大 `ver` 的列，`FINAL` 查詢自動排除 `is_deleted = 1`。
- `index_granularity = 1024` 降低 listing 每頁讀取的列數，primary index 記憶體約數 GB，可接受。
- Partition 讓 `FINAL` 可依 partition 平行去重（`do_not_merge_across_partitions_select_final = 1`），並限制 merge 範圍；bucket 過濾靠主鍵。

5.2 Delimiter listing 用表

```
CREATE TABLE meta.object_by_parent ON CLUSTER mds
(
    cluster_id LowCardinality(String), bucket LowCardinality(String),
    parent String, object_key String,
    size UInt64, etag FixedString(16), mtime DateTime64(3),
    storage_class LowCardinality(String),
    ver UInt64, is_deleted UInt8
)
ENGINE = ReplicatedReplacingMergeTree(
    '/clickhouse/tables/{shard}/meta/object_by_parent', '{replica}', ver,
is_deleted)
PARTITION BY (cluster_id, cityHash64(bucket) % 16)
ORDER BY (cluster_id, bucket, parent, object_key)
SETTINGS index_granularity = 1024;

CREATE TABLE meta.dir_children ON CLUSTER mds
```

```
(
    cluster_id LowCardinality(String), bucket LowCardinality(String),
    parent String, child_prefix String,      -- parent 'a/b/' → child
'a/b/c/'
    ver UInt64, is_deleted UInt8
)
ENGINE = ReplicatedReplacingMergeTree(
    '/clickhouse/tables/{shard}/meta/dir_children', '{replica}', ver,
is_deleted)
ORDER BY (cluster_id, bucket, parent, child_prefix);
```

- `dir_children` 的 MV 以 `arrayJoin` 展開每一層祖先：`a/b/c/d.txt` 產生 `('', 'a/')`、`('a/', 'a/b/')`、`('a/b/', 'a/b/c/')`。
- 目錄只在新增 object 時建立，空目錄由 F5 每日清理；只支援 `delimiter=/`。

5.3 Access 系列表

```
CREATE TABLE access.events_raw ON CLUSTER mds_access
(
    ts          DateTime64(3),
    cluster_id  LowCardinality(String),
    bucket      LowCardinality(String),
    object_key  String,
    op          LowCardinality(String),   -- GET / HEAD / PUT / DELETE / LIST
    status      UInt16,
    bytes       UInt64,
    is_range    UInt8,
    is_internal UInt8,
    principal   LowCardinality(String),
    client_ip   IPv6        TTL toDateTime(ts) + INTERVAL 7 DAY,
    request_id  String      TTL toDateTime(ts) + INTERVAL 7 DAY
)
ENGINE = ReplicatedMergeTree(
    '/clickhouse/tables/{shard}/access/events_raw', '{replica}')
PARTITION BY toYYYYMMDD(ts)
ORDER BY (cluster_id, bucket, ts)
TTL toDateTime(ts) + INTERVAL 30 DAY;

CREATE TABLE access.object_daily ON CLUSTER mds_access
(
    cluster_id LowCardinality(String), bucket LowCardinality(String),
    object_key String, day Date,
    reads      SimpleAggregateFunction(sum, UInt64),
    heads      SimpleAggregateFunction(sum, UInt64),
    bytes_read SimpleAggregateFunction(sum, UInt64),
    last_ts    SimpleAggregateFunction(max, DateTime64(3))
```

```
)
ENGINE = ReplicatedAggregatingMergeTree(
    '/clickhouse/tables/{shard}/access/object_daily', '{replica}')
PARTITION BY toYYYYMM(day)
ORDER BY (cluster_id, bucket, object_key, day)
TTL day + INTERVAL 400 DAY;

CREATE TABLE access.object_profile ON CLUSTER mds_access
(
    cluster_id LowCardinality(String), bucket LowCardinality(String),
    object_key  String,
    first_ts    SimpleAggregateFunction(min, DateTime64(3)),
    last_ts     SimpleAggregateFunction(max, DateTime64(3)),
    total_reads SimpleAggregateFunction(sum, UInt64),
    total_bytes SimpleAggregateFunction(sum, UInt64)
)
ENGINE = ReplicatedAggregatingMergeTree(
    '/clickhouse/tables/{shard}/access/object_profile', '{replica}')
PARTITION BY (cluster_id, cityHash64(bucket) % 16)
ORDER BY (cluster_id, bucket, object_key);
```

- `mds_access` 是只含每個 shard 的 2 個 replica（serving replica 2 與 analytics replica）的 cluster 定義，access 表不放在 serving replica 1，節省空間。
- 聚合表 MV 只計 `op IN ('GET','HEAD') AND status < 400 AND is_internal = 0`。
- `prefix_daily` 以 `arrayJoin` 展開 depth 1–3 的 prefix，欄位為 reads、bytes_read、`uniqCombinedState(object_key)`。
- AggregatingMergeTree 在 merge 前同一 key 可能有多列，讀取時一律 `GROUP BY` 或 `FINAL`。

5.4 Sharding

- Distributed sharding key = `cityHash64(cluster_id, bucket)`，同一 bucket 的 metadata、access、features 落在同一 shard。
- 超大 bucket（> 1B objects）改以 `cityHash64(cluster_id, bucket, top_level_prefix)` 分散，Query Service 依 prefix 定位 shard；跨 top-level prefix 的 listing 由 Query Service 做 k-way merge。

6. Listing Query Service（F4）

Query Service 是 stateless 的 S3 listing shim：把 S3 參數轉成固定模板的參數化 SQL，只在 bucket 所屬 shard 的 serving replica 上執行；delimiter listing 透過「只讀一層目錄」的兩張表，讓成本與子樹大小無關。

6.1 無 delimiter

```
SELECT object_key, size, hex(etag) AS etag, mtime, storage_class
FROM meta.object_meta FINAL
WHERE cluster_id = {cluster:String}
  AND bucket     = {bucket:String}
  AND object_key >= {prefix:String}
  AND object_key <  {prefix_upper:String}   -- prefix 最後一個 byte + 1；
prefix 為空則省略
  AND object_key >  {after:String}          -- start-after 或 token 的
last_key
ORDER BY object_key
LIMIT {max_keys_plus_one:UInt32}
SETTINGS do_not_merge_across_partitions_select_final = 1,
         optimize_read_in_order = 1,
         max_threads = 4,
         max_execution_time = 2;
```

- 用明確範圍取代 `LIKE 'prefix%'`，確保使用主鍵；多取 1 筆判斷 `IsTruncated`。
- `FINAL` 成本與該 range 重疊的 active part 數成正比，part 數是 listing latency 的關鍵指標（第 10 節）；`FINAL` + `optimize_read_in_order` 能否在 LIMIT 後提前結束，Phase 0 以實際版本驗證。

6.2 `delimiter=/`

以 `prefix = 'a/b/'` 為例，平行送出兩個查詢，再依 key 順序 merge 取前 `max_keys` 筆：

```
-- Contents
SELECT object_key, size, hex(etag) AS etag, mtime, storage_class
FROM meta.object_by_parent FINAL
WHERE cluster_id = {cluster:String} AND bucket = {bucket:String}
  AND parent = 'a/b/' AND object_key > {after:String}
ORDER BY object_key LIMIT {max_keys_plus_one:UInt32};

-- CommonPrefixes
SELECT child_prefix
FROM meta.dir_children FINAL
WHERE cluster_id = {cluster:String} AND bucket = {bucket:String}
  AND parent = 'a/b/' AND child_prefix > {after:String}
ORDER BY child_prefix LIMIT {max_keys_plus_one:UInt32};
```

Prefix 不以 `/` 結尾時（例如 `a/b/fo`），`parent` 取到最後一個 `/`（`a/b/`），再加上 `>= 'a/b/fo' AND < 'a/b/fp'`。

6.3 Pagination

- Token = base64(HMAC-signed `{v, cluster, bucket, prefix, delimiter, last_key, index_epoch}`)，client 視為 opaque。

- 無跨頁 snapshot，每頁讀當下狀態，與 S3 原生行為一致；bucket 重新 bootstrap 時 `index_epoch` 遞增，舊 token 仍以 `last_key` 繼續。

6.4 Workload 隔離

項目	Serving（listing）	Analytics（F6、報表）
執行位置	每 shard 的 replica 1、2	每 shard 的 replica 3
Settings profile	`max_threads = 4`、`max_execution_time = 2 s`、`max_memory_usage = 2 GB`	`max_threads = 32`、`max_execution_time = 1 h`、`max_memory_usage = 64 GB`
併發上限	每節點 200	每節點 10
使用者	`mds_query`	`mds_analytics`、`mds_tiering`

所有 replica 都承受 ingest 與 background merge，serving replica 以 `background_pool_size` 等設定限制 merge 對 IO 的影響。

6.5 擴充 API（gRPC + REST gateway）

Endpoint	用途	實作
`PrefixStats`	Prefix 的 object 數、bytes、tier 分佈	`object_meta FINAL` 聚合，加範圍限制
`AccessProfile`	Object / prefix 的 last access 與讀取趨勢	`object_profile`、`object_daily`、`prefix_daily`
`Search`	size、mtime、content_type、tag、storage_class 條件查詢	參數化 SQL，analytics profile
`Export`	大 prefix 全量匯出（類似 S3 Inventory）	`INSERT INTO FUNCTION s3('…/inventory/…', 'Parquet') SELECT …`
`Watermark`	各 cluster / bucket 的 freshness	Kafka consumer lag 與 `max(ingest_ts)`
Admin API	Bucket onboarding、重建、強制 reconcile、tiering kill switch	寫入 registry 與 etcd，全部記 audit log

6.6 Fallback

- Circuit breaker：ClickHouse 錯誤率、listing p99 > 2 s、bucket lag > 5 min 時，自動改走 MinIO。

- Bucket 為 `BOOTSTRAPPING` / `DEGRADED`、delimiter 不是 `/`、或 client 帶 `x-mds-consistency: strong` 時，一律走 MinIO。
- 熱門首頁查詢做 10 s 本地快取，吸收儀表板類的重複輪詢。

7. Access analytics 與 tiering 決策（F6）

決策以 object 為單位分析，以「prefix rule 優先、object manifest 補充」的方式執行：每天為 5B 個 object 計算 access 特徵，套用 policy 與成本模型產生候選，再把同一 prefix 下幾乎全部可搬的候選合併成 prefix rule 交給 SAL。這樣保留 object 層級的精準度，也避免對數十億個小檔逐一下 transition 指令。

7.1 Object 特徵

特徵	定義	來源
`size`、`storage_class`	目前大小與所在 tier	`object_meta`
`age_days`	now − mtime	`object_meta`
`idle_days`	now − last access；從未被讀則為 min(age_days, F3 收集天數)	`object_profile`
`reads_7d` / `reads_30d` / `reads_90d`	GET + HEAD 次數（不含 internal）	`object_daily`
`bytes_read_30d`	讀取 bytes，辨識大物件的 range read	`object_daily`
`residency_days`	進入目前 tier 的天數	`tiering.actions` 與 transition 事件

特徵表每日在 analytics replica 上重建，各 shard 本地執行；三張來源表同 shard 且主鍵順序一致，使用 `full_sorting_merge`，記憶體用量與資料量無關：

```
INSERT INTO tiering.object_features
SELECT
    m.cluster_id, m.bucket, m.object_key, m.size, m.storage_class,
    dateDiff('day', m.mtime, now())                          AS age_days,
    if(p.last_ts IS NULL,
       least(dateDiff('day', m.mtime, now()), {collect_days:UInt32}),
       dateDiff('day', p.last_ts, now()))                    AS idle_days,
    ifNull(d.reads_7d, 0)  AS reads_7d,
    ifNull(d.reads_30d, 0) AS reads_30d,
    ifNull(d.reads_90d, 0) AS reads_90d,
    ifNull(d.bytes_30d, 0) AS bytes_read_30d
FROM
```

```
(
    SELECT cluster_id, bucket, object_key, size, storage_class, mtime
    FROM meta.object_meta FINAL
) AS m
LEFT JOIN
(
    SELECT cluster_id, bucket, object_key, max(last_ts) AS last_ts
    FROM access.object_profile
    GROUP BY cluster_id, bucket, object_key
) AS p USING (cluster_id, bucket, object_key)
LEFT JOIN
(
    SELECT cluster_id, bucket, object_key,
           sumIf(reads + heads, day >= today() - 7)  AS reads_7d,
           sumIf(reads + heads, day >= today() - 30) AS reads_30d,
           sum(reads + heads)                        AS reads_90d,
           sumIf(bytes_read, day >= today() - 30)    AS bytes_30d
    FROM access.object_daily
    WHERE day >= today() - 90
    GROUP BY cluster_id, bucket, object_key
) AS d USING (cluster_id, bucket, object_key)
SETTINGS join_algorithm = 'full_sorting_merge', join_use_nulls = 1;
```

7.2 Tier 與成本模型

Tier	媒介（假設）	儲存成本	讀取代價	單一 object transition 代價
Hot	NVMe / SSD MinIO	C_hot（最高）	低延遲	—
Warm	HDD MinIO（ILM remote tier）	C_warm	延遲較高，經 hot 轉發	一次 GET + PUT + metadata 更新
Cold	高 parity HDD 或歸檔層	C_cold（最低）	可能需 restore，分鐘級以上	同上，restore 另計

每個 demote 候選計算淨效益，`net_benefit > margin` 才成為候選：

$$\text{net_benefit} = \text{size} \times (C_{src} - C_{dst}) \times H - \widehat{\text{reads}}_{H} \times P_{dst} - T_{obj}$$

H 為評估期（例如 90 天），預估讀取次數由 reads_30d 外推，P_dst 為目的 tier 每次讀取的代價（延遲換算或 restore 成本），T_obj 為單一 object 的 transition 固定成本。參數由 storage 團隊依硬體與 SLA 填入 `tiering.policy` 表，修改需審核並保留版本。

7.3 初始 policy（可調）

動作	條件	說明
Hot → Warm	idle_days ≥ 30、reads_30d = 0、age_days ≥ 30、size ≥ 128 KiB、residency_days ≥ 14	小於 128 KiB 只能透過 prefix rule 一起搬
Warm → Cold	idle_days ≥ 90、reads_90d = 0、size ≥ 1 MiB、residency_days ≥ 30	Cold restore 代價高，門檻較嚴
Warm → Hot	reads_7d ≥ 10，或 bytes_read_30d ≥ 5 × size	頻繁讀取或大量 range read
Cold → Hot	reads_7d ≥ 3	每次讀取都需 restore，少量讀取即值得回升
排除	Object Lock / legal hold、tag `tier=pinned`、bucket 不在 allowlist	永不自動搬移

- **遲滯**：搬移後至少停留 `residency_days` 才可再搬；demote 門檻遠高於 promote 的反向門檻，避免振盪。
- **觀測窗口不足**：F3 收集未滿 30 天時所有 demote 一律 dry-run；warm → cold 需滿 90 天。

7.4 從候選到執行

1. **候選**：寫入 `tiering.candidates(run_id, cluster_id, bucket, object_key, action, size, net_benefit, reason)`。
2. **Prefix 合併**：對 depth ≤ 4 的 prefix 計算「候選 bytes / 總 bytes」與「候選數 / 總數」，兩者都 ≥ 95% 且無排除項時合併為一條 prefix rule。
3. **Object manifest**：未合併的候選只保留 size ≥ 1 MiB 者，以 `INSERT INTO FUNCTION s3(…, 'Parquet')` 輸出 manifest 到 MinIO；小檔留待下次評估。
4. **交付 SAL**：Prefix rule 以 append-only immutable rule 寫入 SAL 的 etcd；manifest 交給 SAL executor 批次執行。
5. **回流**：SAL 執行紀錄寫入 `tiering.actions`；MinIO `ObjectTransition` 事件經 F2 更新 `storage_class`。

7.5 安全機制

- **預設 dry-run**：新 policy 或新 bucket 先 dry-run 至少 2 個週期。
- **Blast radius**：每日搬移量 ≤ 來源 tier 容量 2%、單一 bucket ≤ 20%、單次 run object 數上限，超過需人工核准。
- **Canary**：新 policy 先套用於 canary bucket，mis-tier 率達標再擴大。
- **Kill switch**：Admin API 一個 etcd 開關停止所有新 rule 下發。

- **Rollback**：依 `run_id` 從 `tiering.actions` 產生反向 promote manifest。

7.6 效果衡量

指標	定義	目標
Mis-tier 率	Demote 後 30 天內被讀的 bytes / demote 總 bytes	< 2%
Ping-pong 率	90 天內被 demote 又 promote 的 object 比例	< 0.5%
Hot tier 命中率	讀取 bytes 中由 hot tier 服務的比例	依 SLA
釋放容量	每週由 hot 移出的淨 bytes	依容量規劃

每週自動產出報表作為調整依據；累積半年資料後，評估以讀取預測模型取代固定門檻。

8. 一致性模型

Metadata 提供 bounded-staleness 的 eventual consistency：單一 key 依 `ver` 單調前進、已刪除物件不會復活、差異 24 h 內由 F5 收斂。Access 資料是「近似但偏保守」的統計：可能略多計，不會少計經過 gateway 的讀取。

8.1 保證與不保證

項目	是否保證	機制
同一 key 不退回舊狀態	是	ReplacingMergeTree 依 `ver` 保留最新列
已刪除物件不因亂序事件復活	是	Tombstone 保留 ≥ 7 天 + late event 門檻 24 h
Listing 無重複 key	是	一律 `FINAL`
新鮮度可觀測	是	`x-mds-watermark`
最終收斂	是（≤ 24 h）	F5 reconciler
跨頁一致 snapshot	否	與 S3 原生相同
Read-your-writes	否	需要時以 `x-mds-consistency: strong` 走 MinIO
Listing 中的物件一定可 GET	否	刪除可能尚未反映
空目錄立即消失	否	F5 每日清理
Access 次數精確	否	At-least-once，可能略多計

8.2 Freshness 與 watermark

- **Metadata lag** = MinIO 事件時間到 `object_meta` 可查的時間差：notification（秒級）+ Kafka + flush（7.5 s），目標 p99 < 60 s。
- **Bucket watermark** = 該 cluster topic 所有 partition 中已寫入 ClickHouse 的最舊未處理事件時間，Query Service 每 10 s 更新。
- **Access lag** = gateway log 到聚合表可查的時間差：Vector 批次 + Kafka + flush，目標 p99 < 15 min；F6 每日批次，對此不敏感。

8.3 Tombstone 生命週期

1. 刪除事件寫入 `is_deleted = 1` 的列，`FINAL` 查詢立即排除。
2. Tombstone 至少保留 7 天，擋住亂序到達的舊 PUT；late event 門檻為 24 h，因此任何被接受的事件都還有 tombstone 可比較。
3. 每週離峰對每個 partition 執行 `DELETE FROM meta.object_meta WHERE is_deleted = 1 AND ver < {now − 7d}`（lightweight delete），衍生表同樣處理。不用 `OPTIMIZE … FINAL CLEANUP`，因為它會不分年齡移除所有 tombstone。
4. 同一個週期 job 以 anti-join 清除 `access.object_profile` 中已刪除 object 的列。

8.4 Access 資料的已知偏差

偏差來源	方向	影響	處理
Kafka / Vector 重複送達	多計	極少量	可接受，偏向較晚 demote
繞過 gateway 直連 MinIO	少計	可能誤判為冷	網路層禁止直連；或補接 MinIO audit webhook
Internal principal 標記不完整	多計	冷資料誤判為熱	Principal 清單納入變更管理；dashboard 顯示 top principals
Vector / Kafka 長時間中斷	少計	中斷期間讀取遺失	記錄於 `access.gaps`，該期間不納入 idle 判斷，決策延後一個週期

9. 部署拓撲與容量估算

以第 2 節的假設，ClickHouse 為 4 shard × 3 replica 共 12 節點，每個 replica 約 12 TB（access raw event 佔 2/3）；單頁 listing p99 估計 250–450 ms。所有數字需以 Phase 0 實際資料校正，GET / HEAD 速率是最大的變數。

9.1 部署拓撲

元件	數量與規格	放置	備註
ClickHouse server	4 shard × 3 replica = 12 節點；48 core、256 GB RAM、2 × 7.68 TB NVMe	同 shard 的 3 個 replica 分屬 3 個 rack	r1、r2 = serving；r3 = analytics
ClickHouse Keeper	3 節點，獨立部署	跨 3 rack	不與 server 共機
Kafka	6 broker；`meta.events` 每 cluster 64 partitions、`access.events` 128 partitions、`meta.repair` 16 partitions	跨 3 rack，replication factor 3	保留 3 天
Vector	每台 OpenResty 一個 agent	與 OpenResty 同機	Disk buffer ≥ 6 h
Query Service	6+ pods，HPA	Kubernetes	Stateless
Reconciler / Bootstrap	各 2+ pods	Kubernetes	以 lease 分配 range
Tiering Engine	2 pods（leader election）	Kubernetes	每日 batch
SAL Static Tiering	既有部署	—	etcd + executor
Prometheus + Grafana	既有監控平台	—	收集所有元件 metrics
異地備份目標	另一站點的 MinIO bucket	異地	每日 `BACKUP` access / tiering database

9.2 ClickHouse 儲存（每個 replica，壓縮後）

表	估算基礎	每列	5B objects 時
`meta.object_meta`	5B 列；key 排序後前綴相似	50–70 B	~300 GB
`meta.object_by_parent`	5B 列，欄位較少	40–55 B	~240 GB
`meta.dir_children`	目錄數假設 1–2 億	20–30 B	< 10 GB

表	估算基礎	每列	5B objects 時
`access.events_raw`	平均 50K/s ≈ 43 億列 / 天，30 天	50–60 B	~7.5 TB
`access.object_daily`	每天被讀的不同 object 假設 3 億，400 天	~20 B	~2.4 TB
`access.object_profile`	≤ 5B 列	~35 B	≤ 175 GB
`tiering.object_features`	5B 列 × 2 次 run	~60 B	~600 GB
合計			**~11–12 TB**

Access 表只放在 `mds_access` 的 2 個 replica 上，serving replica 1 只需約 1.2 TB。20B objects 時 metadata 相關表約 4 倍、shard 擴到 8；access 表隨讀取速率而非 object 數成長。

9.3 寫入能力

- F2 peak 100K events/s 扇出 3 張表 = 300K rows/s；F3 peak 200K events/s 扇出 4 張表 = 800K rows/s；分到 4 shard 約每 shard 280K rows/s，在大批次寫入下遠低於上限。
- 主要風險是 part 數：每次 flush 在每個被觸及的 partition 產生一個 part，需確認 merge 跟得上（第 10 節告警）。

9.4 單頁 listing latency 預算（max-keys = 1000）

階段	p50	p99
OpenResty 路由 + SigV4	3 ms	5 ms
ClickHouse 解析與規劃	3 ms	10 ms
主鍵定位 + 讀 granule（多個 part）	20–50 ms	150–300 ms
FINAL 去重 + 排序	10–30 ms	50–100 ms
XML 序列化 + 網路	10 ms	30 ms
合計	**50–100 ms**	**250–450 ms**

9.5 50x 是否成立

- 冷大 prefix 與 delimiter listing：MinIO 單頁常為秒級以上，ClickHouse 約 100 ms 級，50x 預期可達。
- 全量 enumeration：`Export` 平行掃描主鍵範圍直接寫 Parquet，1 億 keys 預估 1 分鐘內。
- 熱的小 prefix：MinIO metacache 命中時已在 100 ms 內，不以此驗收。

9.6 每日 F6 計算

每 shard 以 `full_sorting_merge` 處理約 12.5 億列 metadata 與對應 access 聚合，預估 10–30 分鐘；候選與 prefix 合併再 5–10 分鐘，全部在 analytics replica 上執行。

10. 可靠性、監控與運維

Metadata 可由 MinIO 重建，故障最壞結果是 listing 退回 MinIO 原生速度；access 歷史無法重建，是唯一需要真正保護的資料；tiering 決策錯誤的代價最高，因此防護層數最多。

10.1 Failure modes

故障	影響的資料流	影響	處理
Kafka 不可用	F2、F3	Metadata 暫存 MinIO `queue_dir`，access 暫存 Vector buffer；lag 上升	恢復後自動補送；listing lag 超標時 fallback；access 中斷期間記入 `access.gaps`
`queue_dir` / Vector buffer 滿溢	F2、F3	部分事件遺失	Metadata 由 F5 補正；access gap 期間不納入 idle 判斷
Kafka engine consumer 卡住	F2 或 F3	單一 topic lag 上升	`kafka_handle_error_mode = 'stream'` 隔離壞訊息；versioned Protobuf schema
Part 數過多	F2、F3、F4	Insert 被節流；listing `FINAL` 變慢	加大 flush 間隔與 batch、調整 merge pool、暫停 bootstrap
Keeper 失去多數	F2、F3	Replicated 表唯讀，ingest 停止；listing 仍可讀	Keeper 跨 rack；恢復後 Kafka 從 offset 續讀
單一 ClickHouse 節點故障	F4、F6	該 shard 剩餘 replica 承接	Query Service 自動切換；新節點自動從其他 replica 複製
整個 shard 失效	F4、F2、F3	該 shard bucket 的 listing 退回 MinIO；寫入堆積在 Kafka	3 replica 跨 rack；Kafka 保留 3 天
Tiering policy 錯誤	F6、F1	大量資料被誤搬，應用延遲上升	Dry-run、blast radius、canary、kill switch、rollback

故障	影響的資料流	影響	處理
站點失效	全部	全部不可用	Metadata 可重建；access / tiering 由異地備份還原

10.2 關鍵指標

- **F2 / F3 ingest**：各 topic consumer lag、`max(ingest_ts)` 與 now 的差距、`kafka_errors`、每秒寫入列數、MinIO `queue_dir` 深度、Vector buffer 使用率。
- **Merge 健康度**：每 partition active part 數、merge 佇列、`DelayedInserts` / `RejectedInserts`。
- **F4 listing**：p50 / p99（分 delimiter 與否、冷熱 prefix）、fallback 比例、serving replica 併發數。
- **F5 正確性**：drift rate、24 h 內完成比對的 range 比例、late event 數。
- **F3 資料品質**：每小時事件數與歷史同期比較、internal 佔比、top principals。
- **F6 tiering**：每日候選數與 bytes、實際執行量、mis-tier 率、ping-pong 率、run 耗時。

10.3 告警門檻（初始值）

條件	等級
Metadata lag p99 > 5 min 持續 5 min	Page
Access lag > 1 h，或每小時事件數低於歷史同期 50%	Page
任一 partition active part 數 > 300	Page
Drift rate > 1 per million	Ticket；> 100 per million 則 page 並暫停該 bucket listing
Listing fallback 比例 > 5% 持續 10 min	Ticket
單次 tiering run 超過 blast radius 上限	自動暫停並 page
Mis-tier 率週平均 > 5%	Ticket，檢討 policy

10.4 運維操作

- **Schema 變更**：以 `ALTER TABLE … ON CLUSTER` 向後相容地加欄位；不相容變更以新表 + 雙寫切換。
- **Bucket 管理**：onboarding、offboarding、重建、強制 reconcile 皆透過 Admin API，記錄 audit log。

- **備份**：每日 `BACKUP` access 與 tiering database 到異地 MinIO，保留 30 天；metadata database 不備份。
- **升級**：以 replica 為單位 rolling upgrade，先 analytics replica，觀察一週再 serving replica。

11. 取捨、風險與需重新檢視的條件

ClickHouse-only 是以「listing latency 較高且較不穩定」換取「object 層級分析能力」與「只維運一套儲存系統」。這個取捨在 tiering 是核心、listing 可接受數百 ms 的前提下成立；前提改變時需重新評估。

11.1 方案比較

方案	Listing	Object 層級 tiering 分析	維運	結論
ClickHouse-only（選用）	p99 數百 ms，受 part 數影響	原生支援 5B × 數十億事件的 join 與聚合	Kafka + ClickHouse + Keeper	符合目前優先順序
TiKV + ClickHouse	p99 數十 ms	同左	多一套 TiKV / PD，團隊不熟	維運成本最高
Elasticsearch + ClickHouse	p99 ~100 ms	同左	兩套大型叢集	Listing 較好，硬體成本高
Elasticsearch-only	p99 ~100 ms	只能做 prefix 粒度	一套	不符合 object 層級需求

11.2 主要風險與緩解

風險	可能性	緩解
`FINAL` 在 part 數多時使 listing p99 > 500 ms	中	`index_granularity = 1024`、控制 flush 與 partition 數、part 數告警；Phase 0 先 benchmark
Background merge 與 listing 搶 IO	中	Serving / analytics replica 分離、限制 merge pool、NVMe
空目錄延遲消失	高（設計如此）	文件化為已知行為；F5 每日清理

風險	可能性	緩解
Access 收集遺漏直連流量	中	網路層禁止繞過 gateway；或補接 MinIO audit webhook
錯誤 policy 造成大量誤搬	低，但影響大	Dry-run、canary、blast radius、kill switch、rollback
ClickHouse 版本行為差異（`is_deleted`、lightweight delete、read-in-order + FINAL）	中	鎖定 LTS；依賴的行為列入整合測試
MinIO transition 事件不完整	中	F5 reconcile 會比對 storage class；Phase 0 實測

11.3 其他取捨

- **Kafka engine vs 獨立 ingest 服務**：Kafka engine 少一個服務，但錯誤處理與觀測性較弱；若 Phase 2 發現難以排查，改用 ClickHouse Kafka Connect Sink（支援 exactly-once）。
- **Gateway log vs `s3:ObjectAccessed:*` notification**：Gateway log 不增加 MinIO 負載，也不受 `queue_dir` 限制；代價是看不到繞過 gateway 的流量。
- **固定門檻 vs 預測模型**：先以可解釋的固定門檻上線，方便審核與除錯；半年後再評估預測模型。
- **Strong consistency**：需在 PUT 路徑同步寫 metadata，會讓 ClickHouse 進入 F1 的 critical path，刻意不做。

11.4 需要重新檢視的條件

- Listing 實測 p99 長期 > 500 ms，或 QPS 需求 > 1K page/s：在 ClickHouse 前加一層 serving store（TiKV 或 ES），ClickHouse 回到純分析角色。
- Object 數 > 20B，或 access raw event 每日 > 1 TB：增加 shard，並評估對 HEAD 取樣或縮短 raw 保留期。
- 後端改為自帶 filer metadata store 的系統（例如 SeaweedFS）：listing 改由其 metadata store 提供，本系統專注 F3 與 F6。
- 出現跨 region 部署需求：重新設計 shard 放置與 access 資料匯集。

12. 分階段落地與驗收

F3（access 收集）最先上線，因為 access 歷史無法回補，越早累積，F6 越早能從 dry-run 進入正式執行；F2、F4、F5 在觀測期內並行建置。每個階段都有可量化的 exit criteria。

1. **Phase 0 — Baseline 與技術驗證**
    - 以 warp 與自製 list harness（可延伸既有 poc-storage-bench）量測 MinIO 在熱 / 冷、有無 delimiter、不同 prefix 規模下的 listing 與 enumeration。

- 以 10 億列合成資料在目標 ClickHouse LTS 上驗證：`FINAL` listing p50 / p99 與 part 數的關係、`is_deleted` 與 lightweight delete 行為、`full_sorting_merge` 耗時。
- 實測 MinIO `ObjectTransition`、ILM expiration、replication 是否發出 notification。
- 統計實際 GET / HEAD 速率與 internal principal 清單，校正第 9 節。
- Exit：baseline 定案，50x 目標寫成具體數字；合成資料上 listing p99 < 500 ms。

2. **Phase 1 — F3 上線（觀測期開始）**
   - 部署 Vector、Kafka `access.events`、`access.*` 表與 MV，涵蓋所有 gateway；建立資料品質 dashboard。
   - Exit：連續 7 天無 gap，事件數與 OpenResty 日誌總數差異 < 0.1%。
3. **Phase 2 — F2 + F5 與 pilot bucket**
   - 部署 `meta.events` / `meta.repair`、`meta.*` 表、Reconciler、Bootstrap；選一個 1 億級 LOSF bucket bootstrap 並即時同步。
   - Shadow listing：同一請求同時查 ClickHouse 與 MinIO 比對。
   - Exit：metadata lag p99 < 60 s、drift rate < 1 per million、shadow 差異全可由 lag 或空目錄解釋。
4. **Phase 3 — F4 上線與全量 bootstrap**
   - OpenResty 路由、Query Service、circuit breaker、workload 隔離；1% → 10% → 100% 切流量，其餘 bucket 依序 bootstrap。
   - Exit：冷大 prefix 與 enumeration 達 50x、listing p99 < 500 ms、fallback < 1%。
5. **Phase 4 — F6 dry-run → canary → production**
   - F3 累積滿 30 天後每日 dry-run，人工檢視候選與 prefix 合併。
   - Canary bucket 開啟 hot → warm，觀察 30 天 mis-tier 率；累積滿 90 天後開啟 warm → cold 與 promote。
   - Exit：mis-tier 率 < 2%、ping-pong 率 < 0.5%、blast radius 與 rollback 演練通過。
6. **Phase 5 — 擴展**
   - 全部 bucket 納入 tiering、`Export` / `Search` 等擴充 API、跨 cluster 統一 namespace、versioned bucket。
   - 累積半年資料後評估讀取預測模型。